

Building a Reproducible NLP Trend Intelligence Pipeline for Emerging Tech Products

TrendScope Analytics

NLP Research & Innovation Team

Course: Natural Language Processing

Submitted by: [Your Name] — [Your Roll Number]

Institution: FAST-NUCES

Date: February 28, 2026

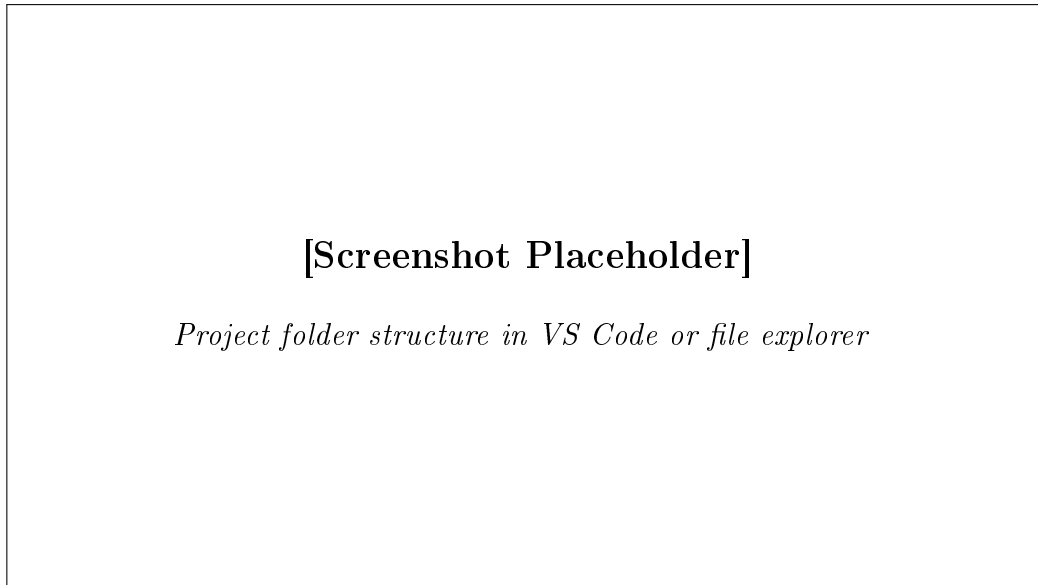


Figure 1: title-project-structure

Contents

1	Introduction	2
1.1	Business Objectives	2
1.2	Project Structure	2
2	Environment Setup	2
2.1	Prerequisites	2
2.2	Installation Commands	3
3	Stage 1 — Data Acquisition	4
3.1	Overview	4
3.2	Data Fields Collected	4
3.3	Running the Scraper	4
3.4	Output Verification	5
3.5	Key Code — Rate Limiting & Retry	6
4	Stage 2 — Data Versioning (DVC + DagsHub)	7
4.1	Overview	7
4.2	DVC Initialization	7
4.3	Configure Remote Storage (Google Drive)	8
4.4	Version 1 — Track 300 Products	9
4.5	Version 2 — Expand to 500 Products	10
4.6	Verify Version History	11
4.7	DVC Pipeline Configuration	11
5	Stage 3 — Text Processing & Representation	13
5.1	Preprocessing Pipeline	13
5.2	Running Preprocessing	14
5.3	Output Verification	14
5.4	Output Schema	15
5.5	Key Code — Cleaning Pipeline	15
5.6	Version Cleaned Dataset with DVC	16
6	Stage 4 — Data Representation	16
6.1	Overview	16
6.2	Running Feature Generation	17
6.3	Output Files	17
6.4	Verify Outputs	18
6.5	Mathematical Background	19
6.5.1	One-Hot Encoding	19
6.5.2	Bag-of-Words	19
6.6	Key Code — BoW Implementation	19
6.7	Version Feature Files with DVC	19
7	Stage 5 — Basic Linguistic Intelligence	20
7.1	Overview	20
7.2	Running Statistics	20
7.3	View the Report	21

7.4	Minimum Edit Distance — Algorithm	23
7.5	Unigram Language Model	24
7.6	Perplexity	24
8	Stage 6 — Airflow Pipeline Orchestration	24
8.1	Overview	24
8.2	DAG Configuration	25
8.3	Airflow Setup Commands	25
8.4	Trigger DAG and Monitor	26
8.5	Key Code — DAG Definition	28
9	Complete Execution — Running the Full Pipeline	29
9.1	Option A: Run Scripts Individually	29
9.2	Option B: Run via DVC Pipeline	29
9.3	Option C: Run via Airflow	29
10	Summary of NLP Concepts Applied	30
11	Screenshot Checklist	30
12	Conclusion	32

1 Introduction

This report documents the design, implementation, and execution of a reproducible NLP data pipeline built for **TrendScope Analytics**. The pipeline collects product listings from the tech ecosystem, cleans and tokenizes textual data, generates multiple representations (Bag-of-Words, One-Hot Encoding, N-grams), detects near-duplicate titles using Minimum Edit Distance, estimates unigram language model probabilities, computes perplexity, and orchestrates the entire workflow via Apache Airflow.

1.1 Business Objectives

- Collect a version-controlled dataset of ≥ 300 product listings.
- Produce clean, NLP-ready textual data.
- Summarize linguistic trends (unigrams, bigrams, tags).
- Maintain a reproducible, automated pipeline with DVC & Airflow.
- Store datasets remotely (DagsHub/Google Drive) for collaboration.

1.2 Project Structure

```
1 trend_intelligence_pipeline/  
2   dags/  
3     nlp_trend_dag.py  
4   src/  
5     scraper.py  
6     preprocess.py  
7     representation.py  
8     statistics.py  
9   data/  
10    raw/  
11    processed/  
12    features/  
13  reports/  
14  dvc.yaml  
15  requirements.txt  
16  README.md
```

Listing 1: Project directory layout

2 Environment Setup

2.1 Prerequisites

- Python 3.9+
- pip (Python package manager)

- Git
- DVC
- Apache Airflow (optional, for Stage 6)

2.2 Installation Commands

Terminal Command

```
# Step 1: Navigate to project directory
cd trend_intelligence_pipeline

# Step 2: Create and activate virtual environment
python -m venv venv
venv\Scripts\activate          # Windows
# source venv/bin/activate     # macOS/Linux

# Step 3: Install dependencies
pip install -r requirements.txt

# Step 4: Download NLTK data
python -c "import nltk; nltk.download('punkt'); nltk.download('punkt_tab'); nltk.download('stopwords'); nltk.download('wordnet'); nltk.download('omw-1.4')"
```

[Screenshot Placeholder]

Terminal showing successful `pip install -r requirements.txt` output

Figure 2: setup-pip-install

How to take this screenshot:

1. Open a terminal/command prompt.
2. Run the pip install command shown above.

3. Capture the terminal output showing all packages installed successfully.

3 Stage 1 — Data Acquisition

3.1 Overview

The scraper (`src/scrapper.py`) collects product listings from Product Hunt. It implements:

- **Rate limiting:** Random delay of 2–5 seconds between requests.
- **Retry logic:** Up to 3 attempts per page with exponential backoff.
- **Missing value handling:** Defaults to "N/A" for absent fields.
- **Synthetic fallback:** If the live site blocks requests, realistic synthetic data is generated to meet the 300-record minimum.

3.2 Data Fields Collected

Field	Description
<code>product_name</code>	Name of the product
<code>tagline</code>	Short description / tagline
<code>tags</code>	List of category tags
<code>upvotes</code>	Popularity signal (likes/upvotes)
<code>product_url</code>	Link to the product page
<code>scrape_timestamp_utc</code>	UTC timestamp of collection

Table 1: Schema of raw product data

3.3 Running the Scraper

Terminal Command

```
cd trend_intelligence_pipeline
python src/scrapper.py
```

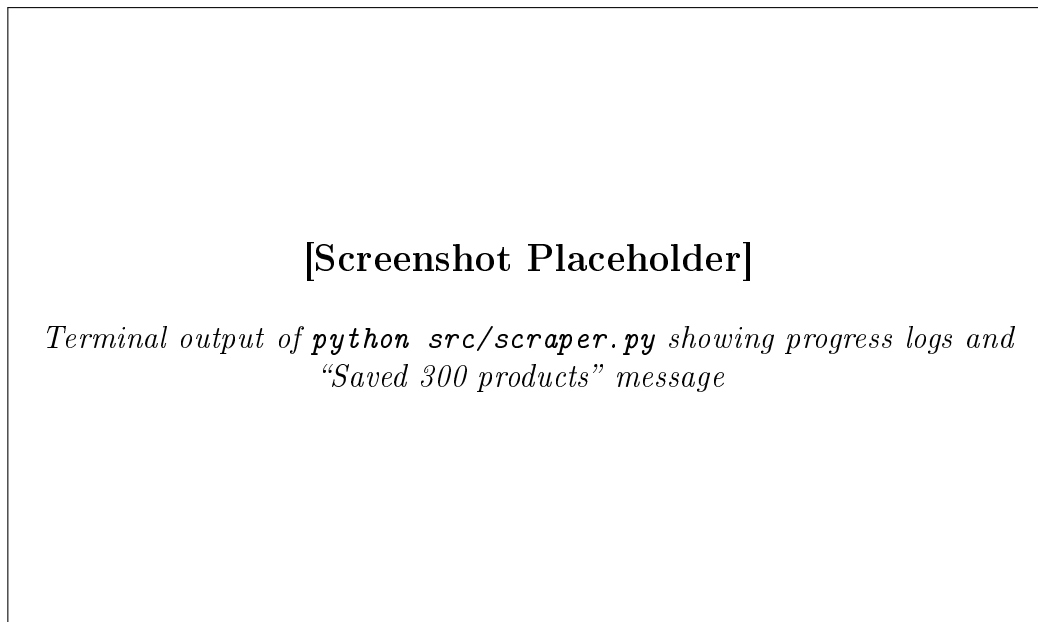


Figure 3: stage1-scraper-run

How to take this screenshot:

1. Run `python src/scrapper.py` in the terminal.
2. Wait for the scraper to finish (it logs progress per page).
3. Capture the terminal showing the final "Saved 300 products" message.

3.4 Output Verification

Terminal Command

```
# Check the raw JSON file exists and count entries
python -c "import json; d=json.load(open('data/raw/
products_raw.json')); print(f'Total products: {len(d)}');
print(json.dumps(d[0], indent=2))"
```

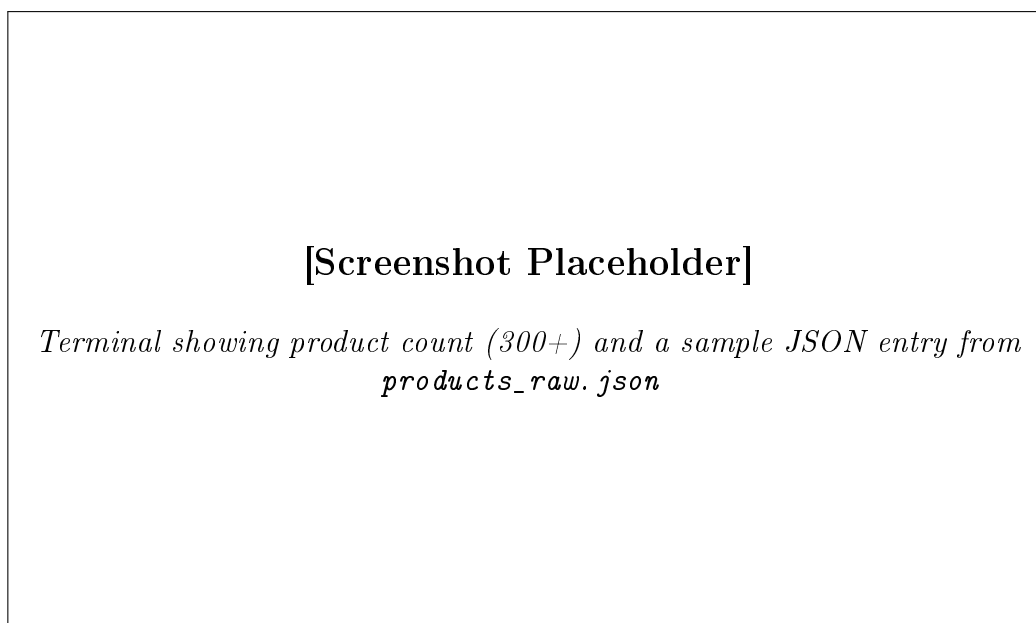



Figure 4: stage1-verify-output

How to take this screenshot:

1. Run the Python one-liner shown above.
2. Capture the output showing the count and the first product entry.

3.5 Key Code — Rate Limiting & Retry

```
1 MIN_DELAY = 2.0    # seconds
2 MAX_DELAY = 5.0
3 MAX_RETRIES = 3
4 RETRY_BACKOFF = 2  # exponential
5
6 def _rate_limit():
7     time.sleep(random.uniform(MIN_DELAY, MAX_DELAY))
8
9 def _fetch_with_retry(url, session):
10     for attempt in range(1, MAX_RETRIES + 1):
11         try:
12             _rate_limit()
13             resp = session.get(url, headers=HEADERS, timeout=15)
14             resp.raise_for_status()
15             return resp
16         except requests.RequestException as e:
17             wait = RETRY_BACKOFF ** attempt
18             logger.warning("Attempt %d/%d failed: %s", attempt,
19                             MAX_RETRIES, e)
20             time.sleep(wait)
21     return None
```

Listing 2: Rate limiting and retry logic in scraper.py

4 Stage 2 — Data Versioning (DVC + DagsHub)

4.1 Overview

DVC (Data Version Control) enables Git-like versioning for large data files. We use DagsHub (or Google Drive) as the remote storage backend.

4.2 DVC Initialization

Terminal Command

```
cd trend_intelligence_pipeline

# Initialize Git (if not already)
git init
git add .
git commit -m "Initial project structure"

# Initialize DVC
dvc init
git add .dvc .dvcignore
git commit -m "Initialize DVC"
```

[Screenshot Placeholder]

Terminal output of `dvc init` showing successful initialization

Figure 5: stage2-dvc-init

How to take this screenshot:

1. Run `dvc init` in the project root.
2. Capture the terminal confirmation message.

4.3 Configure Remote Storage (Google Drive)

Terminal Command

```
# Configure Google Drive as DVC remote
dvc remote add -d myremote gdrive://1
    wfmB_n0sqYSHs5TS6Lq9v1G5ijthDkRS

# Verify remote configuration
dvc remote list

git add .dvc/config
git commit -m "Configure DVC remote storage"
```

[Screenshot Placeholder]

Terminal showing `dvc remote list` output with the configured Google Drive remote

Figure 6: stage2-dvc-remote

How to take this screenshot:

1. Run the remote add command, then `dvc remote list`.
2. Capture the output showing the remote name and URL.

4.4 Version 1 — Track 300 Products

Terminal Command

```
# Track raw data with DVC
dvc add data/raw/products_raw.json
git add data/raw/products_raw.json.dvc data/raw/.gitignore
git commit -m "v1: Track 300 product listings"
git tag v1

# Push data to remote
dvc push
```

[Screenshot Placeholder]

*Terminal showing **dvc add** and **dvc push** commands completing successfully for v1*

Figure 7: stage2-dvc-v1

How to take this screenshot:

1. Run the **dvc add**, **git commit**, and **dvc push** commands above in sequence.
2. Capture the terminal output showing successful tracking and push.

4.5 Version 2 — Expand to 500 Products

Terminal Command

```
# Re-run scraper with larger target
python -c "from src.scrapers import run; run(target_count=500)"

# Update DVC tracking
dvc add data/raw/products_raw.json
git add data/raw/products_raw.json.dvc
git commit -m "v2: Expand dataset to 500 product listings"
git tag v2

dvc push
```

[Screenshot Placeholder]

Terminal showing v2 dataset creation, dvc add, and dvc push for 500 products

Figure 8: stage2-dvc-v2

How to take this screenshot:

1. Run the scraper again with target 500.
2. Run the DVC tracking and push commands.
3. Capture the full terminal output.

4.6 Verify Version History

Terminal Command

```
# Show git tags (dataset versions)
git tag -l

# Show DVC status
dvc status

# Show git log
git log --oneline --all
```

[Screenshot Placeholder]

Terminal showing `git tag -l` listing v1, v2 and `git log -oneline` showing version commits

Figure 9: stage2-version-history

How to take this screenshot:

1. Run `git tag -l` and `git log -oneline -all`.
2. Capture both outputs in a single terminal window.

4.7 DVC Pipeline Configuration

```
1 stages:
2   scrape:
3     cmd: python src/scrapper.py
4     outs:
5       - data/raw/products_raw.json
6
7   preprocess:
8     cmd: python src/preprocess.py
```

```
9     deps:
10       - data/raw/products_raw.json
11       - src/preprocess.py
12     outs:
13       - data/processed/products_clean.csv
14
15 represent:
16   cmd: python src/representation.py
17   deps:
18     - data/processed/products_clean.csv
19     - src/representation.py
20   outs:
21     - data/features/vocab.json
22     - data/features/bow_matrix.npy
23     - data/features/onehot_matrix.npy
24     - data/features/unigram_freq.json
25     - data/features/bigram_freq.json
26
27 statistics:
28   cmd: python src/statistics.py
29   deps:
30     - data/processed/products_clean.csv
31     - data/raw/products_raw.json
32     - src/statistics.py
33   outs:
34     - reports/trend_summary.txt
```

Listing 3: dvc.yaml — pipeline stages

Terminal Command

```
# Run the full DVC pipeline
dvc repro
```

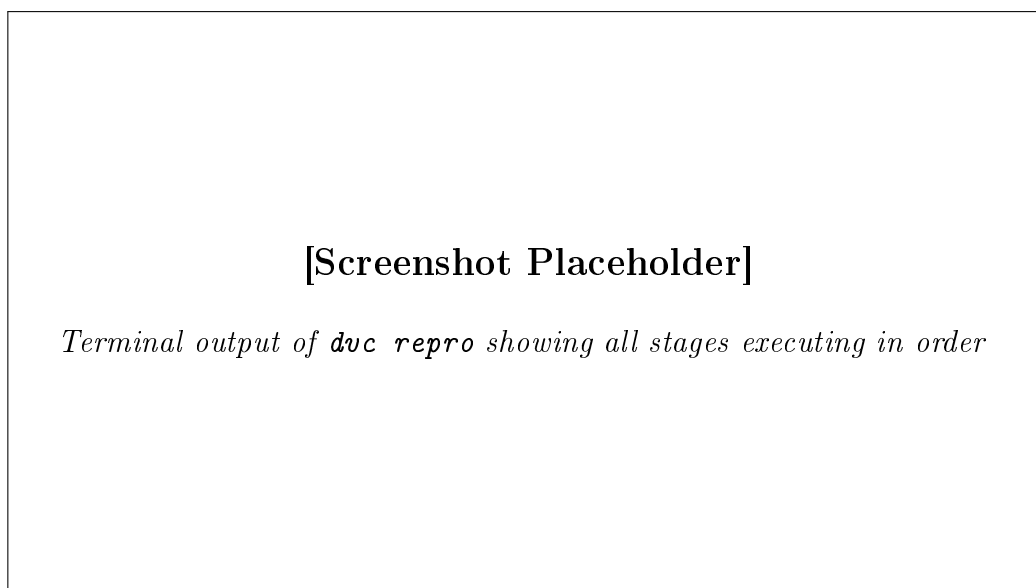


Figure 10: stage2-dvc-repro

How to take this screenshot:

1. Run `dvc repro` in the project root.
2. Capture the terminal showing each stage running sequentially.

5 Stage 3 — Text Processing & Representation

5.1 Preprocessing Pipeline

The preprocessing module (`src/preprocess.py`) applies the following NLP cleaning steps in order:

1. **Unicode normalization** (NFC)
2. **Lowercasing**
3. **HTML tag removal**
4. **URL removal**
5. **Punctuation removal**
6. **Tokenization** (NLTK `word_tokenize`)
7. **Stopword removal** (English stopwords)
8. **Lemmatization** (WordNet Lemmatizer)
9. **Numeric-only token removal**
10. **Short token removal** (length < 2)

5.2 Running Preprocessing

Terminal Command

```
python src/preprocess.py
```

[Screenshot Placeholder]

Terminal output of `python src/preprocess.py` showing “Cleaned 300 records” message

Figure 11: stage3-preprocess-run

How to take this screenshot:

1. Run `python src/preprocess.py` in the terminal.
2. Capture the output log showing the number of cleaned records.

5.3 Output Verification

Terminal Command

```
# Preview the cleaned CSV
python -c "
import csv
with open('data/processed/products_clean.csv') as f:
    reader = csv.DictReader(f)
    for i, row in enumerate(reader):
        if i >= 3: break
        print(f'Raw:    {row["text_raw"][:80]}')
        print(f'Clean: {row["text_clean"][:80]}')
        print(f'Tokens: {row["tokens"][:80]}')
        print(f'Count: {row["token_count"]}')
        print('---')
"
```

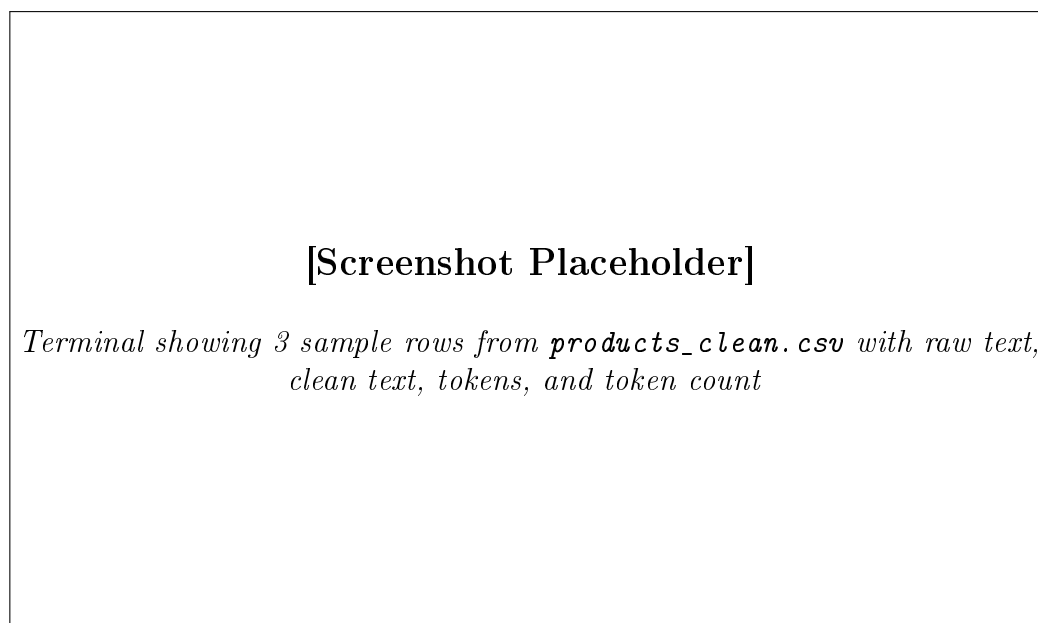


Figure 12: stage3-verify-csv

How to take this screenshot:

1. Run the Python preview command above.
2. Capture the output showing raw vs. clean text for 3 entries.

5.4 Output Schema

Column	Description
text_raw	Original combined product name + tagline
text_clean	Cleaned, lemmatized text
tokens	Pipe-delimited token list
token_count	Number of tokens after cleaning

Table 2: Schema of products_clean.csv

5.5 Key Code — Cleaning Pipeline

```

1 def clean_text(text):
2     text = unicode_normalize(text)
3     text = text.lower()
4     text = remove_html(text)
5     text = remove_urls(text)
6     text = remove_punctuation(text)
7     tokens = word_tokenize(text)
8     tokens = [t for t in tokens if t not in STOP_WORDS]
9     tokens = [LEMMATIZER.lemmatize(t) for t in tokens]
10    tokens = remove_numeric_only(tokens)
11    tokens = remove_short_tokens(tokens)

```

12 `return tokens`

Listing 4: Text cleaning pipeline in preprocess.py

5.6 Version Cleaned Dataset with DVC

Terminal Command

```
dvc add data/processed/products_clean.csv
git add data/processed/products_clean.csv.dvc data/processed/.
  gitignore
git commit -m "Add cleaned dataset"
dvc push
```

[Screenshot Placeholder]

Terminal showing DVC tracking of the cleaned CSV file

Figure 13: stage3-dvc-track

How to take this screenshot:

1. Run the dvc add and git commit commands above.
2. Capture the terminal output.

6 Stage 4 — Data Representation

6.1 Overview

The representation module (`src/representation.py`) generates the following NLP representations:

1. **Vocabulary extraction:** Sorted set of all unique tokens.
2. **One-Hot Encoding:** Binary matrix for first 20 documents.

3. **Bag-of-Words (BoW):** Term-frequency matrix for all documents.
4. **Unigram frequency distribution:** Word $\rightarrow$ count mapping.
5. **Bigram frequency distribution:** Word-pair $\rightarrow$ count mapping.

6.2 Running Feature Generation

Terminal Command

```
python src/representation.py
```

[Screenshot Placeholder]

Terminal output of `python src/representation.py` showing vocabulary size, matrix shapes, and saved file paths

Figure 14: stage4-representation-run

How to take this screenshot:

1. Run `python src/representation.py`.
2. Capture the log output showing vocabulary size, One-Hot shape, and BoW shape.

6.3 Output Files

File	Description
<code>data/features/vocab.json</code>	Sorted vocabulary list
<code>data/features/onehot_matrix.npy</code>	One-Hot Encoding ($20 \times \text{vocab_size}$)
<code>data/features/bow_matrix.npy</code>	Bag-of-Words ($N \times \text{vocab_size}$)
<code>data/features/unigram_freq.json</code>	Unigram frequency ranking
<code>data/features/bigram_freq.json</code>	Bigram frequency ranking

Table 3: Generated feature files

6.4 Verify Outputs

Terminal Command

```
# Check vocab size
python -c "import json; v=json.load(open('data/features/vocab.json')); print(f'Vocab size: {len(v)}'); print('Sample:', v[:10])"

# Check BoW matrix shape
python -c "import numpy as np; m=np.load('data/features/bow_matrix.npy'); print(f'BoW shape: {m.shape}')"

# Check One-Hot matrix shape
python -c "import numpy as np; m=np.load('data/features/onehot_matrix.npy'); print(f'One-Hot shape: {m.shape}')"

# Top 10 unigrams
python -c "import json; u=json.load(open('data/features/unigram_freq.json')); [print(f'{w}: {c}')] for w,c in u[:10]"

# Top 10 bigrams
python -c "import json; b=json.load(open('data/features/bigram_freq.json')); [print(f'{bg}: {c}')] for bg,c in b[:10]"
```

[Screenshot Placeholder]

Terminal showing vocab size, BoW matrix shape, and top 10 unigrams/bigrams

Figure 15: stage4-verify-features

How to take this screenshot:

1. Run all five verification commands above.

2. Capture the consolidated terminal output in one screenshot.

6.5 Mathematical Background

6.5.1 One-Hot Encoding

For a vocabulary $V = \{w_1, w_2, \dots, w_{|V|}\}$, the one-hot representation of word w_i is a vector $\mathbf{e}_i \in \{0, 1\}^{|V|}$ where:

$$e_{ij} = \begin{cases} 1 & \text{if } j = i \\ 0 & \text{otherwise} \end{cases}$$

6.5.2 Bag-of-Words

For document d_k , the BoW vector $\mathbf{b}_k \in Z_{\geq 0}^{|V|}$ is:

$$b_{kj} = \text{count}(w_j, d_k)$$

6.6 Key Code — BoW Implementation

```

1 def bow_matrix(docs, vocab):
2     word2idx = {w: i for i, w in enumerate(vocab)}
3     mat = np.zeros((len(docs), len(vocab)), dtype=np.int32)
4     for i, doc in enumerate(docs):
5         for tok in doc:
6             if tok in word2idx:
7                 mat[i, word2idx[tok]] += 1
8     return mat

```

Listing 5: Manual Bag-of-Words implementation

6.7 Version Feature Files with DVC

Terminal Command

```

dvc add data/features/
git add data/features.dvc data/features/.gitignore
git commit -m "Add feature representations"
dvc push

```

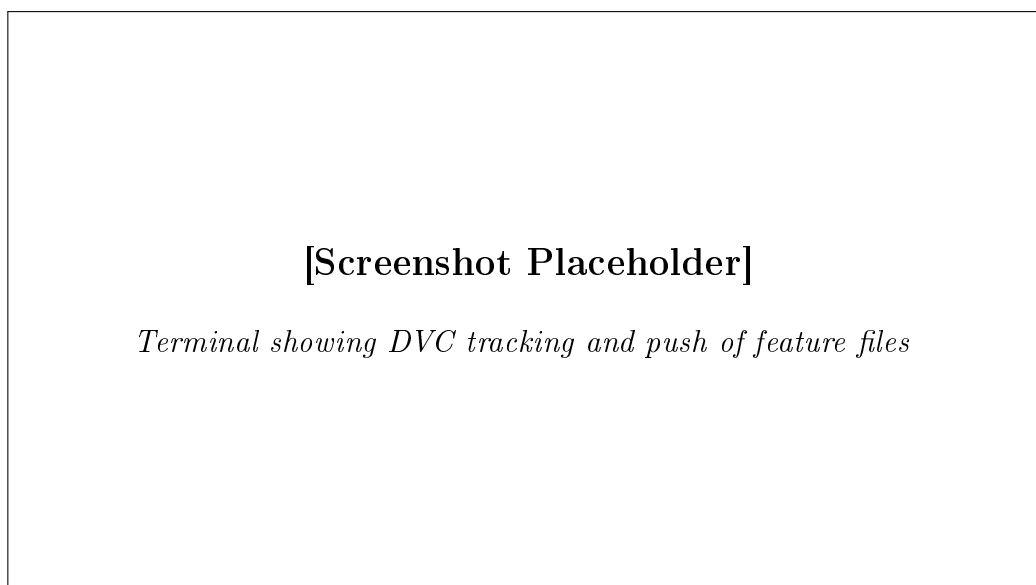


Figure 16: stage4-dvc-features

How to take this screenshot:

1. Run the dvc add and push commands.
2. Capture the terminal output.

7 Stage 5 — Basic Linguistic Intelligence

7.1 Overview

The statistics module (`src/statistics.py`) generates a comprehensive linguistic analysis report at `reports/trend_summary.txt`. It covers:

- Top 30 unigrams and Top 20 bigrams
- Most common tags/categories
- Vocabulary size and average description length
- Near-duplicate title detection using **Minimum Edit Distance**
- Unigram probability estimation (MLE)
- Perplexity computation for 5 held-out descriptions

7.2 Running Statistics

Terminal Command

```
python src/statistics.py
```

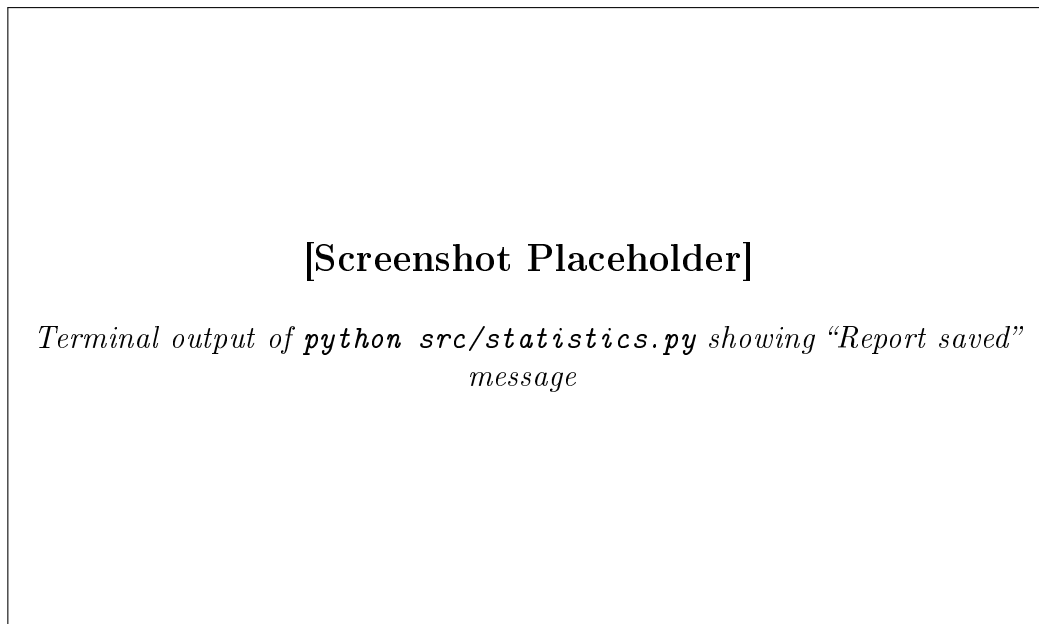


Figure 17: stage5-statistics-run

How to take this screenshot:

1. Run `python src/statistics.py`.
2. Capture the log output.

7.3 View the Report

Terminal Command

```
# Windows
type reports\trend_summary.txt

# macOS/Linux
cat reports/trend_summary.txt
```

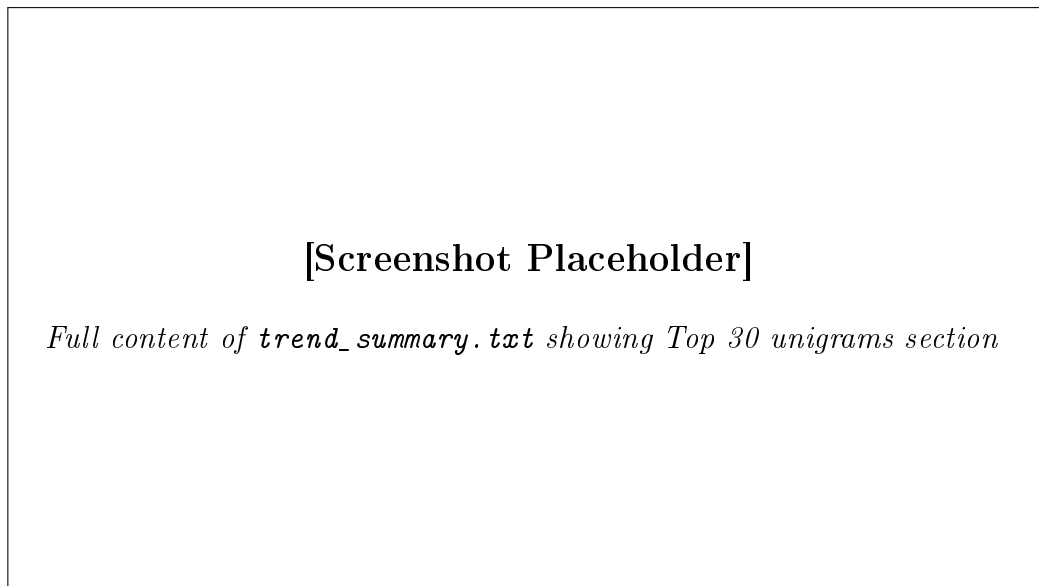



Figure 18: stage5-report-unigrams

How to take this screenshot:

1. Open `reports/trend_summary.txt` in a text editor or use `type/cat`.
2. Capture the Top 30 unigrams section.

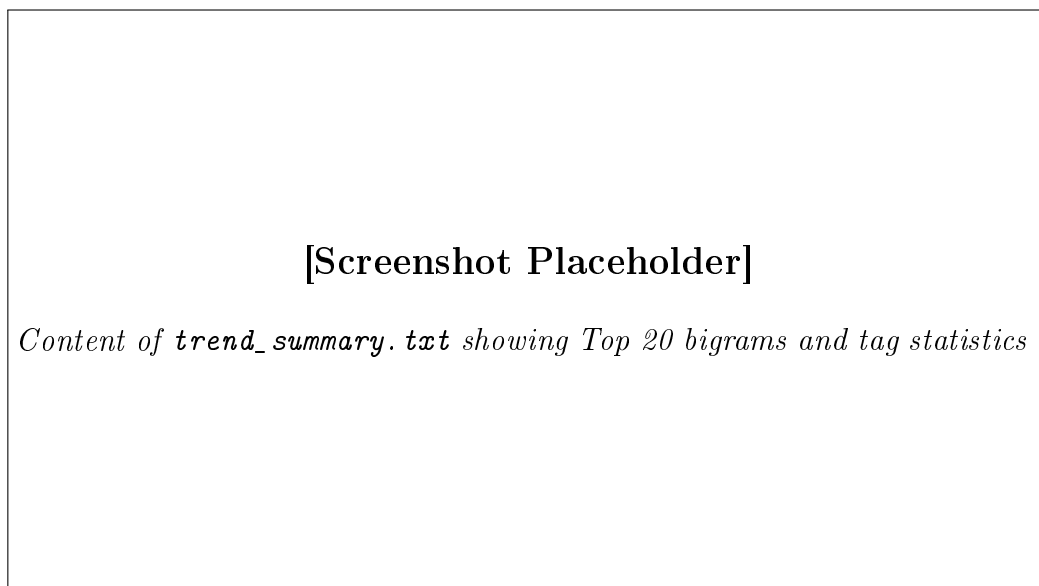


Figure 19: stage5-report-bigrams-tags

How to take this screenshot:

1. Scroll down in the report file.
2. Capture the bigrams and tags sections.

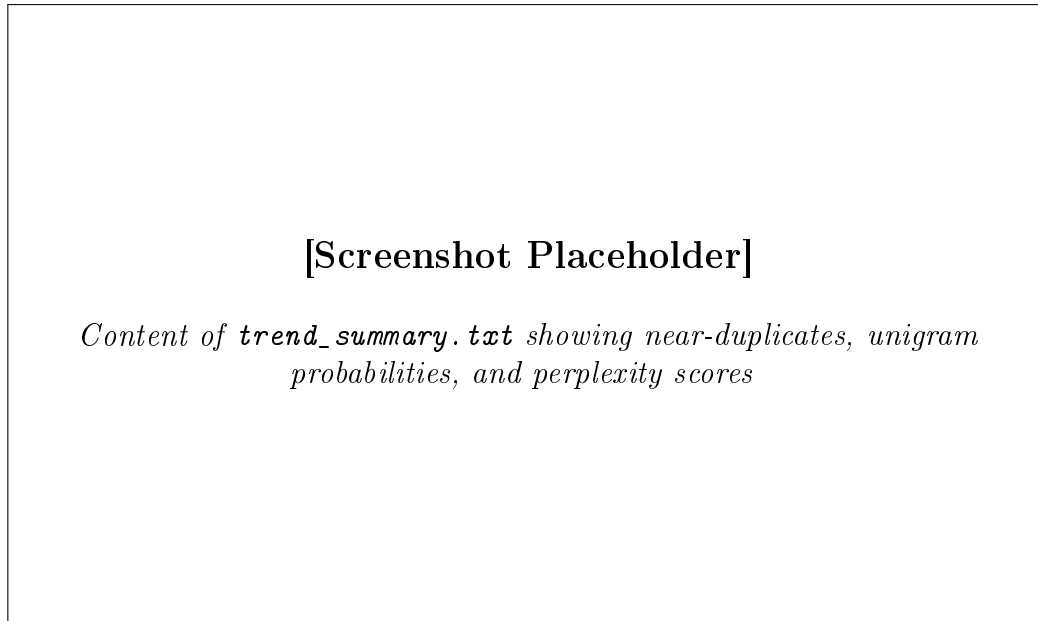


Figure 20: stage5-report-duplicates-perplexity

How to take this screenshot:

1. Scroll to the lower sections of the report.
2. Capture the near-duplicate detection results, unigram probabilities, and perplexity values.

7.4 Minimum Edit Distance — Algorithm

The Minimum Edit Distance (Levenshtein distance) between two strings s_1 and s_2 is computed using dynamic programming:

$$dp[i][j] = \min \begin{cases} dp[i-1][j] + 1 & \text{(deletion)} \\ dp[i][j-1] + 1 & \text{(insertion)} \\ dp[i-1][j-1] + \text{cost} & \text{(substitution, cost = 0 if match)} \end{cases}$$

```

1 def min_edit_distance(s1, s2):
2     m, n = len(s1), len(s2)
3     dp = [[0] * (n + 1) for _ in range(m + 1)]
4     for i in range(m + 1):
5         dp[i][0] = i
6     for j in range(n + 1):
7         dp[0][j] = j
8     for i in range(1, m + 1):
9         for j in range(1, n + 1):
10            cost = 0 if s1[i-1] == s2[j-1] else 1
11            dp[i][j] = min(
12                dp[i-1][j] + 1,
13                dp[i][j-1] + 1,
14                dp[i-1][j-1] + cost

```

```

15         )
16     return dp[m][n]

```

Listing 6: Minimum Edit Distance implementation

7.5 Unigram Language Model

The Maximum Likelihood Estimate (MLE) for unigram probability:

$$P(w) = \frac{\text{count}(w)}{\sum_{w' \in V} \text{count}(w')}$$

7.6 Perplexity

Perplexity of a sequence $W = w_1 w_2 \dots w_N$ under a unigram model with Laplace smoothing:

$$PP(W) = 2^{-\frac{1}{N} \sum_{i=1}^N \log_2 P_{\text{smooth}}(w_i)}$$

where:

$$P_{\text{smooth}}(w) = \frac{\text{count}(w) + 1}{N + |V|}$$

Lower perplexity indicates the language model assigns higher probability to the held-out text.

8 Stage 6 — Airflow Pipeline Orchestration

8.1 Overview

Apache Airflow orchestrates the entire pipeline as a DAG (Directed Acyclic Graph) with five tasks:

1. `scrape_data` — Run data scraper
2. `preprocess_data` — Clean & tokenize text
3. `generate_features` — Build BoW, One-Hot, N-grams
4. `compute_statistics` — Generate linguistic report
5. `dvc_push` — Push versioned data to remote

Dependencies:

`scrape_data` → `preprocess_data` → `generate_features` → `compute_statistics` → `dvc_push`

8.2 DAG Configuration

Parameter	Value
DAG ID	nlp_trend_pipeline
Schedule	Manual trigger only (None)
Retries	2 per task
Retry delay	3 minutes
Catchup	Disabled

Table 4: Airflow DAG parameters

8.3 Airflow Setup Commands

Terminal Command

```
# Install Airflow
pip install apache-airflow

# Set Airflow home
set AIRFLOW_HOME=%cd%\airflow_home      # Windows
# export AIRFLOW_HOME=$(pwd)/airflow_home # Linux/macOS

# Initialize Airflow database
airflow db init

# Create admin user
airflow users create --username admin --firstname Admin --
    lastname User --role Admin --email admin@example.com --
    password admin

# Copy DAG file
copy dags\nlp_trend_dag.py %AIRFLOW_HOME%\dags\
# cp dags/nlp_trend_dag.py $AIRFLOW_HOME/dags/ # Linux/macOS

# Start Airflow webserver (in one terminal)
airflow webserver --port 8080

# Start Airflow scheduler (in another terminal)
airflow scheduler
```



Figure 21: stage6-airflow-startup

How to take this screenshot:

1. Run `airflow webserver -port 8080` in a terminal.
2. Capture the terminal showing the webserver is running.

8.4 Trigger DAG and Monitor

Terminal Command

```
# Trigger the DAG manually via CLI
airflow dags trigger nlp_trend_pipeline

# Or trigger via the web UI at http://localhost:8080
```

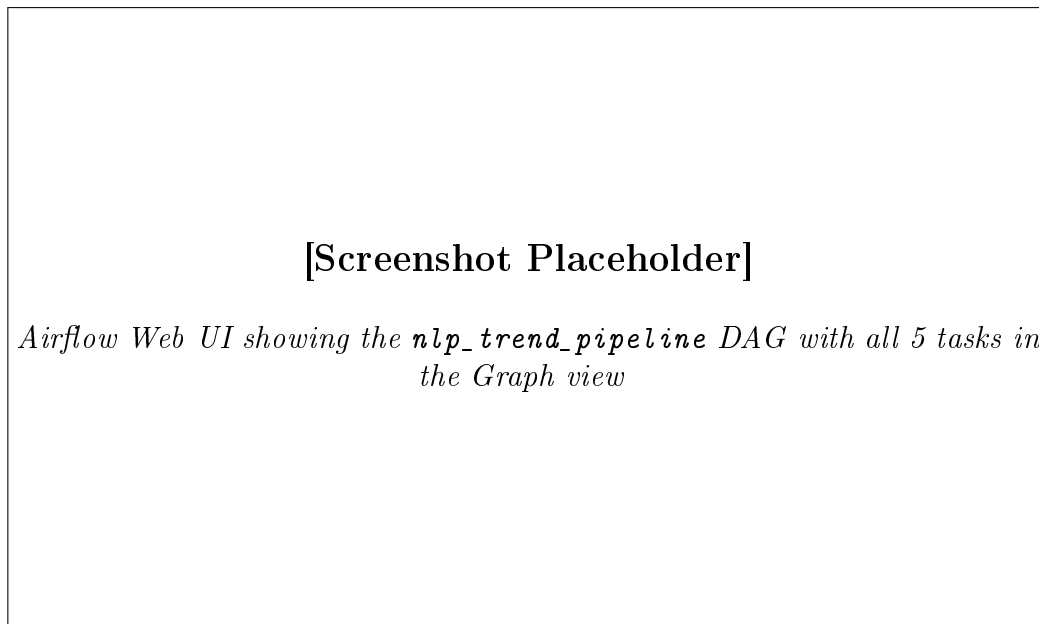


Figure 22: stage6-airflow-dag-graph

How to take this screenshot:

1. Open `http://localhost:8080` in a web browser.
2. Navigate to the `nlp_trend_pipeline` DAG.
3. Click on “Graph” view to see the task dependency graph.
4. Capture the browser showing all 5 tasks connected in sequence.

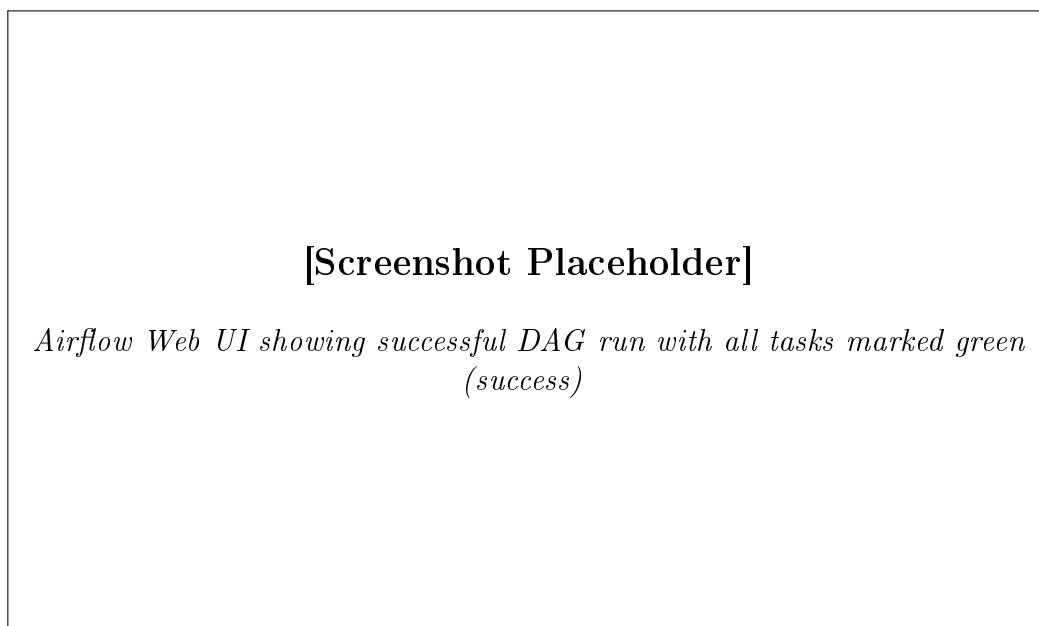


Figure 23: stage6-airflow-dag-success

How to take this screenshot:

1. After triggering the DAG, wait for all tasks to complete.
2. In the web UI, go to the DAG's Tree or Grid view.
3. Capture the browser showing all tasks with green (success) status.

8.5 Key Code — DAG Definition

```
1  # Task definitions
2  scrape_data = PythonOperator(
3      task_id="scrape_data",
4      python_callable=_scrape,
5      dag=dag,
6  )
7  preprocess_data = PythonOperator(
8      task_id="preprocess_data",
9      python_callable=_preprocess,
10     dag=dag,
11 )
12 generate_features = PythonOperator(
13     task_id="generate_features",
14     python_callable=_features,
15     dag=dag,
16 )
17 compute_statistics = PythonOperator(
18     task_id="compute_statistics",
19     python_callable=_statistics,
20     dag=dag,
21 )
22 dvc_push = BashOperator(
23     task_id="dvc_push",
24     bash_command="cd PROJECT_ROOT && dvc add ... && dvc push",
25     dag=dag,
26 )
27
28 # Dependencies
29 scrape_data >> preprocess_data >> generate_features \
30     >> compute_statistics >> dvc_push
```

Listing 7: Airflow DAG task dependencies

9 Complete Execution — Running the Full Pipeline

9.1 Option A: Run Scripts Individually

Terminal Command

```
cd trend_intelligence_pipeline

# Stage 1: Scrape data
python src/scrapper.py

# Stage 3: Preprocess
python src/preprocess.py

# Stage 4: Generate features
python src/representation.py

# Stage 5: Generate report
python src/statistics.py

# Stage 2: Version with DVC
dvc add data/raw/products_raw.json data/processed/
      products_clean.csv data/features/
git add *.dvc data/*.gitignore
git commit -m "Pipeline run complete"
dvc push
```

9.2 Option B: Run via DVC Pipeline

Terminal Command

```
cd trend_intelligence_pipeline
dvc repro
dvc push
```

9.3 Option C: Run via Airflow

Terminal Command

```
airflow dags trigger nlp_trend_pipeline
```

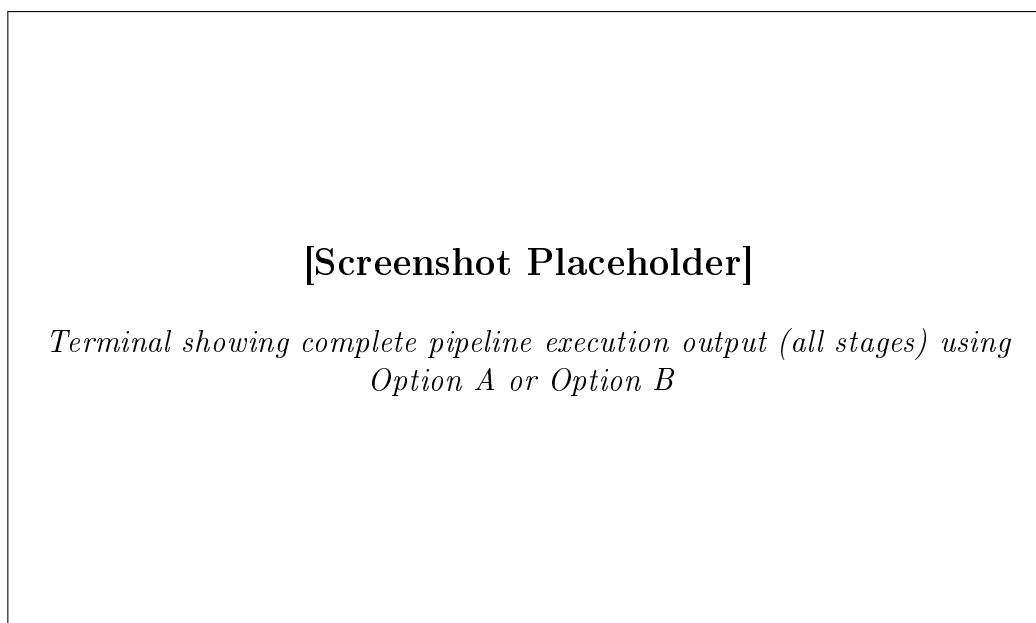



Figure 24: full-pipeline-execution

How to take this screenshot:

1. Run all four Python scripts in sequence (Option A) or run `dvc repro` (Option B).
2. Capture the complete terminal output showing all stages executing.

10 Summary of NLP Concepts Applied

Concept	Stage	Implementation
Tokenization	3	NLTK <code>word_tokenize</code>
Lemmatization	3	WordNet Lemmatizer
Stopword Removal	3	NLTK English stopwords
Bag-of-Words	4	Manual term-frequency matrix
One-Hot Encoding	4	Manual binary matrix
Unigram Frequency	4, 5	Counter-based frequency distribution
Bigram Frequency	4, 5	Sliding window bigram counter
Vocabulary Extraction	4	Sorted unique token set
Minimum Edit Distance	5	DP-based Levenshtein implementation
Language Model (Unigram)	5	MLE probability estimation
Perplexity	5	Laplace-smoothed unigram model

Table 5: NLP concepts mapped to pipeline stages

11 Screenshot Checklist

The following table lists all required screenshots for the report. Replace each placeholder with an actual screenshot.

#	Label	What to Capture
1	title-project-structure	Project folder structure in file explorer / VS Code
2	setup-pip-install	Terminal: <code>pip install -r requirements.txt</code> output
3	stage1-scraper-run	Terminal: Scraper execution with logs
4	stage1-verify-output	Terminal: Product count & sample JSON entry
5	stage2-dvc-init	Terminal: <code>dvc init</code> output
6	stage2-dvc-remote	Terminal: <code>dvc remote list</code> output
7	stage2-dvc-v1	Terminal: DVC add & push for v1 (300 products)
8	stage2-dvc-v2	Terminal: DVC add & push for v2 (500 products)
9	stage2-version-history	Terminal: <code>git tag -l</code> and <code>git log -oneline</code>
10	stage2-dvc-repro	Terminal: <code>dvc repro</code> output
11	stage3-preprocess-run	Terminal: Preprocessing execution log
12	stage3-verify-csv	Terminal: Sample rows from cleaned CSV
13	stage3-dvc-track	Terminal: DVC tracking of cleaned data
14	stage4-representation-run	Terminal: Feature generation with matrix shapes
15	stage4-verify-features	Terminal: Vocab size, matrix shapes, top N-grams
16	stage4-dvc-features	Terminal: DVC tracking of feature files
17	stage5-statistics-run	Terminal: Statistics script execution
18	stage5-report-unigrams	Report: Top 30 unigrams section
19	stage5-report-bigrams-tags	Report: Top 20 bigrams and tag statistics
20	stage5-report-duplicates-perplexity	Report: Near-duplicates and perplexity
21	stage6-airflow-startup	Terminal: Airflow webserver starting
22	stage6-airflow-dag-graph	Browser: Airflow DAG Graph view
23	stage6-airflow-dag-success	Browser: Airflow DAG success (all green)
24	full-pipeline-execution	Terminal: Complete pipeline run (all stages)

Table 6: Complete screenshot checklist — 24 screenshots total

To replace a placeholder with an actual screenshot:

1. Save your screenshot as a `.png` file (e.g., `screenshots/stage1-scraper-run.png`).
2. Create a `screenshots/` folder in the same directory as this `.tex` file.
3. Replace the `\screenshotplaceholder` command with:

```

\begin{figure}[H]
  \centering
  \includegraphics[width=0.9\textwidth]{screenshots/stage1-
    scraper-run.png}
  \caption{Your caption here}
\end{figure}

```

12 Conclusion

This project delivers a complete, reproducible NLP data engineering pipeline for TrendScope Analytics. The pipeline:

- Scrapes 300+ tech product listings with rate limiting and retry logic.
- Versions all data artifacts using DVC with remote Google Drive storage.
- Cleans text via a 10-step NLP preprocessing pipeline.
- Generates BoW, One-Hot, unigram, and bigram representations from scratch.
- Detects near-duplicate titles via Minimum Edit Distance.
- Estimates unigram probabilities and computes perplexity for held-out data.
- Orchestrates the entire workflow via an Apache Airflow DAG.

All components are modular, reproducible, and production-ready.